

Introduction to Robotics

2st Homework

Tasks

1. The frames remain the same as Homework 1.
2. The frame remains the same as Homework 1.
3. This frame had to change from the previous one in order for the Robot Arm to grasp the cube from its starting position (the newton – raphson method fails to find a solution for the previous transformation). This time the end – effector grasps the cube from its upper side, so the transformation matrix between the effector and the cube is the following:

$$T_{\text{eff_cube}} = \begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 2.5 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

This matrix comes from the matrix multiplication of the inverse of the world – effector transformation and the world – cube transformation

$$\text{inv}(T_{w_eff}) * T_{wc} = T_{eff_cube}.$$

The world – effector transformation (T_{w_eff}) was calculated as a rotation of the world frame about the y – axis by pi rads and a translation of

$$(x, y, z) = (17.5, 2.5, 5).$$

4. The two frames now change as follows: (see python code)

a. $T_{\text{cube_out}} = \begin{bmatrix} 0.71 & 0 & 0.71 & 10 \\ -0.71 & 0 & 0.71 & 7.5 \\ 0 & -1 & 0 & 10 \\ 0 & 0 & 0 & 1 \end{bmatrix}$, first the world frame is

rotated about x – axis then about z – axis and finally translated to finally match the desired matrix.

b. $T_{\text{cube_in}} = \begin{bmatrix} 0.71 & 0 & 0.71 & 10 \\ -0.71 & 0 & 0.71 & 12.5 \\ 0 & -1 & 0 & 10 \\ 0 & 0 & 0 & 1 \end{bmatrix}$, it's the same transformation

matrix except a translation along y – axis, so the cube gets placed directly in the middle of the hole.

5. To use inverse kinematics in order to calculate the angle values, we must find the screw axes of the robot structure and calculate its forward kinematics.

a. First step is to find the transformation matrix from robot base to its end –

effector in the zero configuration. So, $M = \begin{bmatrix} 1 & 0 & 0 & 0.8 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 55.38 \\ 0 & 0 & 0 & 1 \end{bmatrix}$.

b. Next we define the screw – axes of the robot (see python code). The screw – axes are:

```
[[0 0 1 0 0 0]]
[[ 0.  0.  1. -0. -0. -0.]]
[[ 0.  1.  0. -16.95 -0. -0. ]]
[[ 0.  0.  1. -0. -0. -0.]]
[[ 0. -1.  0. 28.5 -0. -0. ]]
[[ 0.  0.  1. -0. -0. -0.]]
[[ 0. -1.  0. 41.28 -0. -0. ]]
[[ 0.  0.  1. -0. -0. -0.]]
[[ 0.  0.  1. -0. -0.8 -0. ]]
```

c. Next we calculate the exponential terms through the formula

$\exp(\hat{S}_1\theta_1) \exp(\hat{S}_2\theta_2) \cdots \exp(\hat{S}_N\theta_N)$ (see python code).

- d. Then, we calculate the desired transformation matrix based on the initial values of the joint angles and through the formula

$$\mathbf{T} = \exp(\hat{\mathbf{S}}_1\theta_1) \exp(\hat{\mathbf{S}}_2\theta_2) \cdots \exp(\hat{\mathbf{S}}_N\theta_N) \mathbf{M}$$

- e. We continue by filling the world – Jacobian matrix with the correct vectors, calculated by the formula (see python code)

$$\begin{aligned} \mathbf{J}_{w1} &= \mathbf{S}_1 \\ \mathbf{J}_{w2} &= [\mathbf{Ad}_{\exp(\hat{\mathbf{S}}_1 q_1)}] \mathbf{S}_2 \\ [\mathbf{J}_{w1}(\mathbf{q}) \quad \mathbf{J}_{w2}(\mathbf{q}) \quad \cdots \quad \mathbf{J}_{wn}(\mathbf{q})] \mathbf{J}_{w3} &= [\mathbf{Ad}_{\exp(\hat{\mathbf{S}}_1 q_1) \exp(\hat{\mathbf{S}}_2 q_2)}] \mathbf{S}_3, \text{ the} \end{aligned}$$

Jacobian matrix for initial angles set to zero is this:

$$\begin{bmatrix} [& 0. & 0. & 0. & 0. & 0. & 0. & 0. &] \\ [& 0. & 0. & 1. & 0. & -1. & 0. & -1. & 0. &] \\ [& 1. & 1. & 0. & 1. & 0. & 1. & 0. & 1. &] \\ [& 0. & 0. & -16.95 & 0. & 28.5 & 0. & 41.28 & 0. &] \\ [& 0. & 0. & 0. & 0. & 0. & 0. & 0. & 0. &] \\ [& 0. & 0. & 0. & 0. & 0. & 0. & 0. & 0. &] \end{bmatrix}.$$

We can now implement the inverse kinematics algorithm (in this case using the Newton – Raphson method) to approach the values of the angles within the desired tolerance.

The desired transformation is the matrix $\mathbf{T}_{w_eff} = \begin{bmatrix} -1 & 0 & 0 & 17.5 \\ 0 & 1 & 0 & 2.5 \\ 0 & 0 & -1 & 5 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$

Running the method, the angles of the joints are these:

$$\text{Joint angles (deg): } \begin{bmatrix} 11.66755476 & -1039.18099388 & -5.70382555 & -440.43966713 \\ 4.34875112 & -1858.95499149 & 5.09905844 \end{bmatrix}.$$

The result can be verified with forward kinematics passing these values as initial angle values and calculating the desired transformation matrix. It will be the same as $\mathbf{T}_{w_eff}$.

6. Using the same tools from task 5 but changing the desired transformation matrix, we can compute the angles needed to place the cube in the “placing position”. The new transformation matrix is

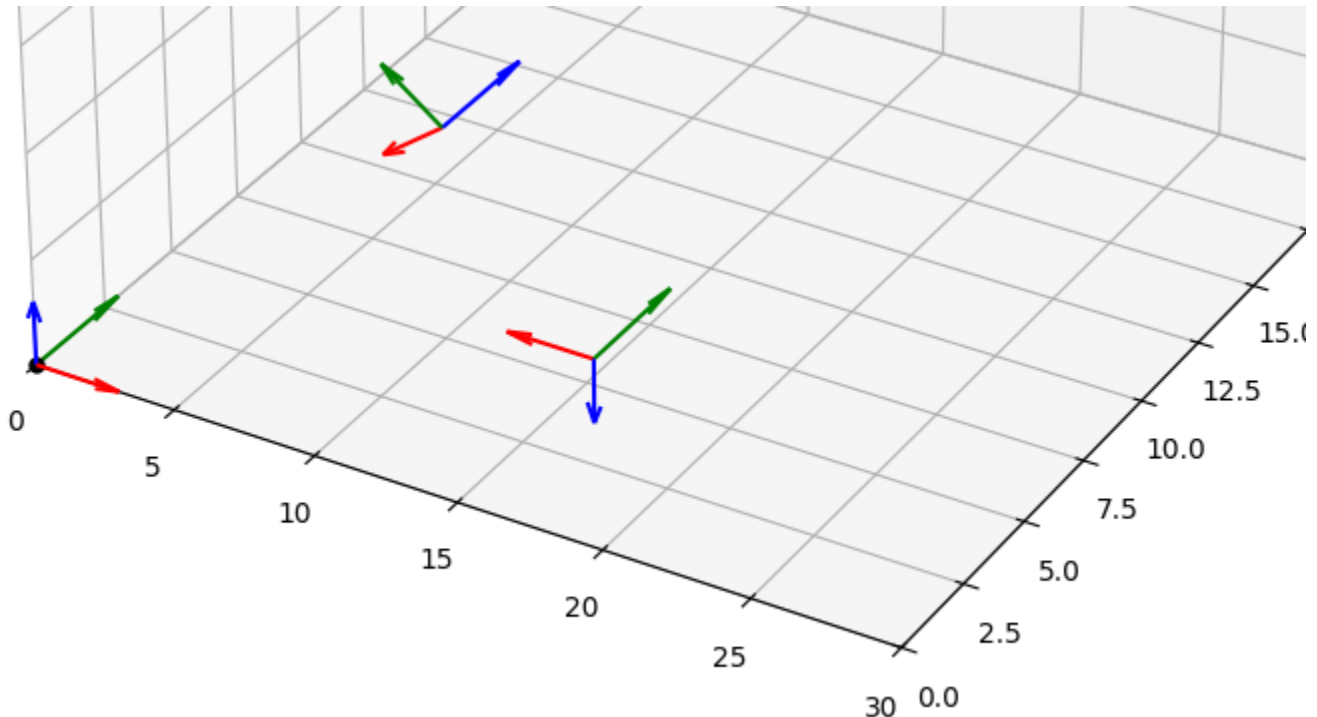
$$T_{w_eff_out} = \begin{bmatrix} -0.71 & 0 & -0.71 & 10 \\ -0.71 & 0 & 0.71 & 5 \\ 0 & 1 & 0 & 10 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \text{ keeping the effector's orientation}$$

towards the cube the same as the start. The angles obtained from the method

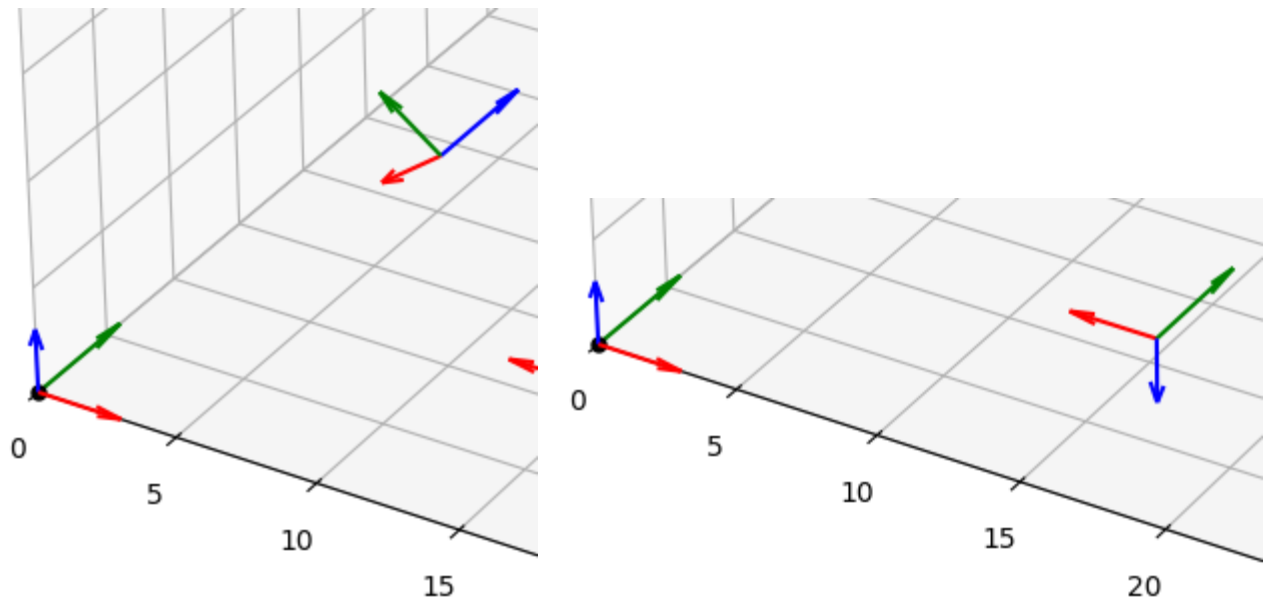
are: `[ 4.59752871 4.59752871 -14.00330119 -15.67247296 11.69948738 -27.53226063 -16.60987376 2.12438462]`.

The result can be again verified by passing the values as the initial angles and seeing that the desired transformation matrix will be the same as the one above.

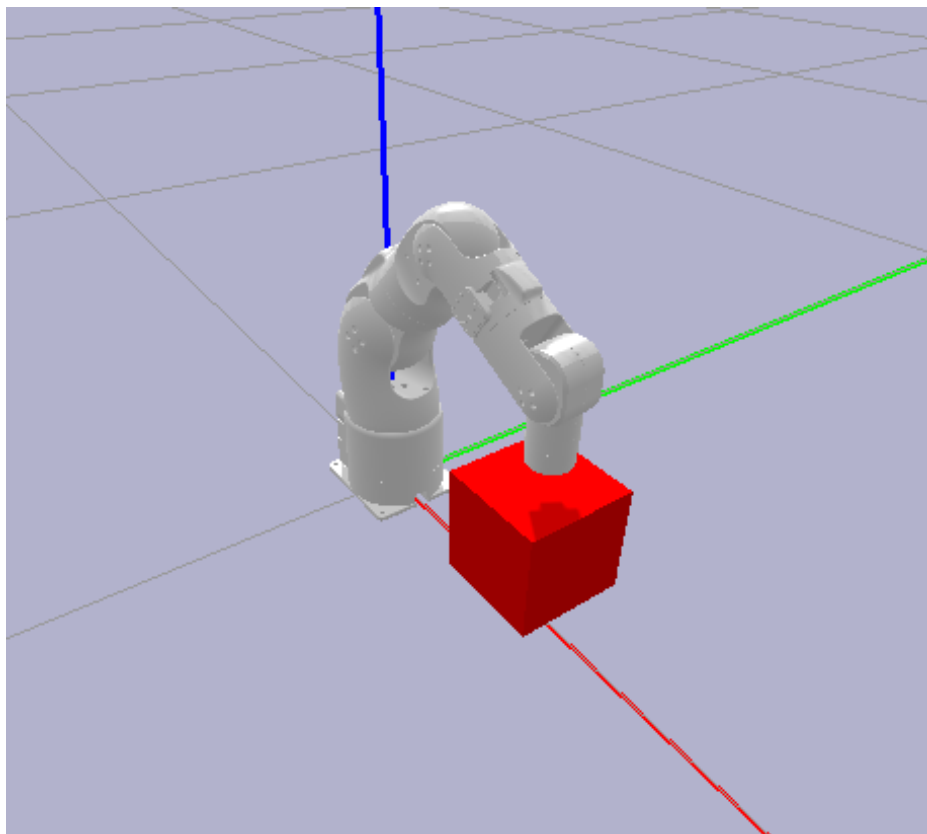
7. First, we visualize the desired frames of the end – effector targets:



Then, the frames obtained by the inverse kinematics method:



The frames are identical, therefore the method calculated the angles accurately. We can also visualize the robot trying to grab the cube in 3D view:



Unfortunately, this viewer does not work with the second position of the robot.

